

MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software and Automation Engineering

Iulian Pavlov, FAF-241

Report

Laboratory work nr. 3

The Command Line: Shell, Pipes and Text Filters

Course: Operating Systems (Track A: Linux)

Checked by:
Zaica Maia, *University Assistant*
Department of Software and Automation Engineering,
FCIM Faculty, UTM

Chişinău – 2026

Contents

1 Theoretical Background and Objectives **2**
1.1 Laboratory Purpose and Competencies . . . 2
1.2 Shell Architecture: Built-ins vs. External Binaries . . . 2
1.3 The Shell Expansion Pipeline and Quoting Rules . . . 3
1.4 UNIX I/O Streams and File Descriptors . . . 4

2 Investigation of Shell Mechanics **4**
2.1 Part 1: Terminal Navigation, Help, and Path Resolution . . . 4
2.1.1 Autocompletion and Command History . . . 4
2.1.2 Command Resolution and the `$PATH` Variable . . . 4
2.1.3 Exit Status of Missing Commands . . . 5
2.1.4 Built-in Help and Man Pages . . . 5
2.2 Part 2: What the Shell Does to Your Line . . . 6
2.2.1 Test Environment Setup . . . 6
2.2.2 Prediction vs. Real Output Verification . . . 6
2.3 Part 3: Redirection, Chaining, and Job Control . . . 8
2.3.1 Separating Standard Output and Standard Error . . . 8
2.3.2 Comparison of Redirection Order . . . 8
2.3.3 Truncation vs. Appending . . . 8
2.3.4 File Argument vs. Standard Input Redirection . . . 8
2.3.5 Command Chaining Operators . . . 9
2.3.6 Job Control and Signal Handling . . . 9

3 Data Processing and Stream Filtering **9**
3.1 Dataset Specification . . . 9
3.2 One-Liner Pipelines (Q1 – Q8) . . . 9
3.3 Bonus Task (+1): Extended Regular Expressions . . . 11
3.4 Complete One-Liners Script (`oneliners.sh`) . . . 12

4 Script Engineering and Task Automation **12**
4.1 Log Analysis Script Implementation (`report.sh`) . . . 12
4.2 Execution and Error Handling Verification . . . 12
4.3 Bonus Task (+1): Automated Scheduling via Cron Daemon . . . 13

5 Answers to Laboratory Questions **13**
5.1 Part 1 Questions . . . 13
5.2 Part 2 Questions . . . 14
5.3 Part 3 Questions . . . 14
5.4 Part 4 Question (Q8) . . . 15
5.5 Part 5 Questions . . . 15

6 Conclusions **16**

1 Theoretical Background and Objectives

1.1 Laboratory Purpose and Competencies

The objective of this laboratory work is to gain in-depth, hands-on mastery over the POSIX/Linux command-line interface (CLI) and the GNU Bash shell. Key competencies developed throughout this laboratory include:

- Understanding the internal execution mechanics of the shell, distinguishing between shell built-in primitives and external binaries residing on disk.
- Mastering path resolution via the `$PATH` environment variable and interpreting process exit statuses (`$?`).
- Rigorous prediction and verification of shell expansion phases, including pathname expansion (globbing), brace expansion, parameter/variable expansion, command substitution, arithmetic evaluation, and word splitting.
- Manipulating standard POSIX I/O streams (`stdin`, `stdout`, `stderr`) via redirection operators (`>`, `»`, `<`, `2>`, `2>&1`) and understanding left-to-right evaluation semantics.
- Chaining command execution deterministically using sequential (`;`) and conditional control operators (`&&`, `||`).
- Process control and asynchronous job management in the background (`&`, `jobs`, `kill`).
- Constructing complex, high-throughput data processing pipelines leveraging the UNIX philosophy of composable filters (`grep`, `cut`, `sort`, `uniq`, `awk`, `wc`, `tee`).
- Writing robust POSIX shell scripts with parameter validation, standardized stream routing, non-zero error signaling, and automated scheduling via the `cron` daemon.

1.2 Shell Architecture: Built-ins vs. External Binaries

When a user submits a command to the shell, the interpreter parses the line and determines whether the commanded symbol corresponds to:

1. An alias defined in the current shell session.
2. A reserved shell keyword (e.g., `if`, `for`, `while`).
3. A shell function defined in the environment.
4. A shell **built-in** (e.g., `cd`, `echo`, `exit`, `pwd`).
5. An **external binary** located by traversing the directories enumerated in `$PATH`.

When executing an external binary, the shell employs the fundamental UNIX process creation model: it invokes the `fork()` system call to clone its own process space, followed by `execve()` within the child process to replace its memory image with the executable binary. In contrast, shell built-ins execute directly within the address space of the parent shell process.

This architectural distinction is of paramount importance for commands that manipulate shell state. Specifically, the `cd` command **must** be a built-in. If `cd` were implemented as an external binary, the kernel would fork a child process, execute `cd` inside that child process, change the child's current working directory (`CWD`), and immediately terminate. The parent shell process, awaiting completion, would remain in its original working directory completely unaffected.

1.3 The Shell Expansion Pipeline and Quoting Rules

Before executing any command line, Bash processes the input through a strict multi-phase expansion pipeline:

1. **Brace Expansion**: Expands arbitrary strings and numerical/character ranges enclosed in braces, such as `{1..5}` into `1 2 3 4 5` or `log{1..3}.txt` into `log1.txt log2.txt log3.txt`.
2. **Tilde Expansion**: Replaces `~` with the user's home directory.
3. **Parameter and Variable Expansion**: Replaces `$VAR` or `${VAR}` with the variable's value.
4. **Command Substitution**: Replaces `$(command)` or `` `command` `` with the standard output generated by executing the enclosed command.
5. **Arithmetic Expansion**: Evaluates mathematical expressions enclosed in `$((expression))`, e.g., `$((7 * 6))` evaluates to `42`.
6. **Word Splitting**: Unquoted results of expansions are split into discrete positional tokens based on delimiters defined by the Internal Field Separator (`IFS`), by default space, tab, and newline.
7. **Pathname Expansion (Globbing)**: Replaces patterns containing wildcards (`*`, `?`, `[...]`) with matching file paths discovered in the filesystem. If no matches exist, Bash preserves the pattern literally.
8. **Quote Removal**: Strips unquoted quote characters (`'`, `"`) that were used to delimit tokens during previous phases.

Quoting rules dictate which expansion stages are suppressed:

- **Single Quotes (`'...'`)**: Strong quoting. Completely suppresses all expansions. Every enclosed character is interpreted strictly literally.
- **Double Quotes (`"..."`)**: Weak quoting. Suppresses globbing and word splitting, but permits variable expansion (`$VAR`), command substitution (`$(...)`), arithmetic expansion (`$((...))`), and backslash escape sequences.
- **Backslash (`\`)**: Escapes the immediately following character, stripping any special shell meaning.

1.4 UNIX I/O Streams and File Descriptors

Every standard UNIX process is spawned with three standard file descriptors (FD):

Table 1: Standard POSIX File Descriptors

File Descriptor	POSIX Name	Stream Name	Default Target
0	`STDIN_FILENO`	Standard Input (`stdin`)	Keyboard / Terminal
1	`STDOUT_FILENO`	Standard Output (`stdout`)	Terminal Screen
2	`STDERR_FILENO`	Standard Error (`stderr`)	Terminal Screen

Redirection operators manipulate these file descriptors before the command begins execution. Redirections are evaluated strictly from left to right:

- `> file`: Redirects file descriptor 1 (stdout) to `file`, truncating it.
- `» file`: Redirects file descriptor 1 to `file`, appending to it.
- `< file`: Redirects file descriptor 0 (stdin) to read from `file`.
- `2> file`: Redirects file descriptor 2 (stderr) to `file`.
- `2>&1`: Duplicates file descriptor 1 into file descriptor 2, routing stderr to wherever stdout is currently directed.

2 Investigation of Shell Mechanics

2.1 Part 1: Terminal Navigation, Help, and Path Resolution

2.1.1 Autocompletion and Command History

The GNU Readline library embedded in Bash provides intelligent autocompletion and interactive history search:

- Typing `ls /usr/sh` and pressing `Tab` twice displayed 227 candidate directories and files starting with `sh` inside `/usr/` (e.g., `share/`, `lib/`, etc.). Because the prefix is ambiguous, the shell emits a bell or prompt displaying all matching options.
- Typing `cd /et` and pressing `Tab` once autocompleted unambiguously to `cd /etc/`, appending the trailing directory slash automatically.
- Pressing `Ctrl+R` engaged Readline's *reverse-i-search* capability, allowing instantaneous recall of previous commands by typing substrings of past invocations.

2.1.2 Command Resolution and the `$PATH` Variable

To inspect whether commands are built into the shell or located on the filesystem, the `type` and `which` utilities were executed:

Listing 1: Inspection of command resolution and binary paths

```
~ > type cd ; type ls ; type echo ; which ls
cd is aliased to `zd'
ls is aliased to `eza -lh --group-directories-first --icons=auto'
echo is a shell builtin
/usr/bin/ls
```

In this environment, `echo` is resolved as an internal shell built-in, while `which ls` identifies the actual physical binary location on disk as `/usr/bin/ls`. The active shell environment configures convenient aliases (`cd` aliased to the zoxide jump utility `zd`, and `ls` aliased to the modern directory listing tool `eza`).

Next, the contents of the `$PATH` search variable were inspected. By piping the string through the stream filter `tr ':' '\n'`, the colon-delimited path list was transformed into one directory per line:

Listing 2: Directory breakdown of the PATH environment variable

```
~ > echo $PATH | tr ':' '\n'
/home/sunflower/.local/share/mise/installs/claude/latest
/home/sunflower/.local/share/mise/installs/codex/latest/bin
/home/sunflower/.local/share/mise/installs/gemini/latest/node_modules/.bin
/home/sunflower/.local/share/mise/installs/gh/latest/gh_2.102.0_linux_amd64/bin
/home/sunflower/.local/share/mise/installs/node/26.8.2/bin
/home/sunflower/.local/share/mise/shims
/usr/share/omarchy/bin
/usr/local/sbin
/usr/local/bin
/usr/bin
/home/sunflower/.local/bin
/usr/bin/site_perl
/usr/bin/vendor_perl
/usr/bin/core_perl
/opt/010editor
```

A total of 15 directories constitute the current search path. When an unqualified command name is entered, the shell queries each directory sequentially from top to bottom. The `ls` binary was discovered inside the 10th search directory: `/usr/bin`.

2.1.3 Exit Status of Missing Commands

Attempting to invoke a non-existent command triggers Bash's command lookup failure:

Listing 3: Non-existent command execution and exit code inspection

```
~ > nosuchprog ; echo "exit code: $?"
bash: command not found: nosuchprog
exit code: 127
```

The exit status `127` is the standardized POSIX return code signaling that the requested command could not be located in any directory specified by `$PATH`.

2.1.4 Built-in Help and Man Pages

To consult documentation without a web browser:

- `man ls`: Opens the system manual pager for `ls`. In standard GNU `ls`, the `-t` option sorts entries by modification time (newest first). In `eza`, this corresponds to `-sort=modified` or `-s date`.

- `man -k "list directory"`: Queries the `mandb` keyword database, returning matching utilities such as `dir (1)`, `ls (1)`, and `vdir (1)`.
- `help cd`: Displays the internal documentation for the shell built-in `cd`, detailing options such as `-L` (follow symbolic links) and `-P` (use physical directory structure).

2.2 Part 2: What the Shell Does to Your Line

2.2.1 Test Environment Setup

A sandbox directory was created with deliberate test filenames:

Listing 4: Creating test files in the sandbox directory

```
~ > mkdir -p ~/os-lab3/play && cd ~/os-lab3/play
~ > touch a.c. b.c file1.txt file2.txt notes.txt "alfa beta.txt"
~ > ls
Permissions Size User      Date Modified Name
.rw-r--r--     0 sunflower  5 Oct 17:07  a.c.
.rw-r--r--     0 sunflower  5 Oct 17:07 'alfa beta.txt'
.rw-r--r--     0 sunflower  5 Oct 17:07  b.c
.rw-r--r--     0 sunflower  5 Oct 17:07  file1.txt
.rw-r--r--     0 sunflower  5 Oct 17:07  file2.txt
.rw-r--r--     0 sunflower  5 Oct 17:07  notes.txt
```

2.2.2 Prediction vs. Real Output Verification

Prior to executing each expansion command, a prediction was recorded. Table 2 documents the comparative analysis:

Table 2: Comprehensive Comparison of Shell Expansion Predictions and Actual Outputs

Command	**Prediction**	**Real Output**	**Technical Explanation**
`echo *.c`	`b.c`	`b.c`	The pattern `*.c` matches filenames ending in `.c`. Since `a.c.` ended with an extra period, only `b.c` satisfied the glob.
`echo file?.txt`	`filetxt1.txt filetxt2.txt`	`file1.txt file2.txt`	The `?` wildcard matches exactly one arbitrary character, matching `file1.txt` and `file2.txt`.
`echo [a-b]*`	`alfabeta.txt`	`a.c. alfa beta.txt b.c`	The character range `[a-b]` matches any filename starting with 'a' or 'b', matching all three files.
`echo *.xyz`	`*.xyz`	`*.xyz`	When no directory entries match a glob pattern, Bash leaves the wildcard pattern unexpanded as a literal string.
`ls alfa beta.txt`	`error`	`"alfa": No such file "beta.txt": No such file`	The shell split the unquoted space into two separate arguments; neither file exists individually.
`ls "alfa beta.txt"`	`home/sunflower`	`'alfa beta.txt'`	Double quotes prevent word splitting, passing the complete filename containing space as a single token.
`echo '$HOME' "$HOME" \$HOME`	`$HOME /home/sunflower $HOME`	`$HOME /home/sunflower $HOME`	Single quotes inhibit variable expansion; double quotes expand `$HOME`; backslash escapes the dollar sign.
`echo {1..5} ; echo $((7 * 6))`	`1 2 3 4 5 42`	`1 2 3 4 5 42`	Brace expansion creates integer sequences; `$((..))` evaluates integer arithmetic expressions.
`echo "today is $(date +%A), I am $(whoami)"`	`today is Monday i am sunflower`	`today is Monday, I am sunflower`	Command substitution `$(...)` captures and embeds standard output of external commands into the string.
`touch log{1..3}.txt && ls log*`	`logs log1 log2 log3`	`log1.txt log2.txt log3.txt`	Brace expansion generated three touch targets; subsequent globbing matched all three created files.

2.3 Part 3: Redirection, Chaining, and Job Control

2.3.1 Separating Standard Output and Standard Error

The command `ls /etc /nope > out.txt 2> err.txt` was executed. Because `/etc` exists and `/nope` does not, the program produces both valid output and error diagnostics:

- File descriptor 1 (`stdout`) received the directory listing of `/etc` and was stored in `out.txt` (totaling 1981 bytes).
- File descriptor 2 (`stderr`) received the error diagnostic: `"/nope": No such file or directory (os error 2)` and was routed into `err.txt`.

2.3.2 Comparison of Redirection Order

To understand file descriptor assignment, two contrasting syntax structures were evaluated:

1. `ls /etc /nope > all.txt 2>&1`: Evaluated from left to right. First, `stdout` (FD 1) is redirected to `all.txt`. Then, `2>&1` duplicates FD 1 into FD 2 (`stderr`). Since FD 1 is already pointing to `all.txt`, FD 2 also points to `all.txt`. Both valid listings and error diagnostics are successfully captured inside `all.txt`.
2. `ls /etc /nope 2>&1 > all.txt`: Evaluated from left to right. First, `2>&1` duplicates FD 1 into FD 2. At that precise moment, FD 1 is still pointing to the terminal screen. Therefore, FD 2 is attached to the terminal screen! Subsequently, `> all.txt` redirects FD 1 to `all.txt`. Consequently, the listing of `/etc` is saved into `all.txt`, while the error diagnostic is printed directly onto the terminal screen.

2.3.3 Truncation vs. Appending

The difference between `>` and `»` was demonstrated using:

```
echo first > notes.md ; echo second >> notes.md ; cat notes.md
```

The single chevron `>` truncated and wrote `first`. The double chevron `»` preserved existing contents and appended `second`, resulting in a two-line file.

2.3.4 File Argument vs. Standard Input Redirection

Evaluating `wc -l < access.log` versus `wc -l access.log` revealed fundamental I/O behavior:

- `wc -l access.log`: `wc` receives the filename as an argument in `argv[1]`. It opens the file itself and prints the line count alongside the filename (`500 access.log`).
- `wc -l < access.log`: The shell opens the file and redirects it to file descriptor 0 (`stdin`) before executing `wc`. The program reads anonymously from `stdin`, completely unaware of any underlying filename, printing only the numerical count (`500`).

2.3.5 Command Chaining Operators

Three distinct execution chaining operators were analyzed:

- `;` (Sequential): Executes commands unconditionally in sequence regardless of success or failure.
- `&&` (Logical AND): Executes the second command if and only if the first command terminates with an exit status of `0` (success).
- `||` (Logical OR): Executes the second command if and only if the first command terminates with a non-zero exit status (failure).

Testing `ls /nope 2> /dev/null || echo "failed with code $?"` suppressed the error output to `/dev/null` and triggered the fallback block, printing `failed with code 2`.

2.3.6 Job Control and Signal Handling

Executing `sleep 100 &` launched a persistent process in the background. Running `jobs` confirmed the active background task:

```
[1]+ Running sleep 100 &
```

The process was terminated gracefully using `kill %1`, which sent the `SIGTERM` signal to background job 1, transitioning its state to `Terminated`.

3 Data Processing and Stream Filtering

3.1 Dataset Specification

A synthetic web server access log (`access.log`) comprising exactly 500 lines was generated. Each record adheres to the following whitespace-separated schema:

`[Date]` `[Time]` `[User]` `[Status]` `[Page]`

Example record: `2026-10-05 10:14 user3 FAIL /page2`

3.2 One-Liner Pipelines (Q1 – Q8)

Q1. How many requests failed?

```
grep -c FAIL access.log
```

Output: `100`
Explanation: The `-c` flag counts the exact number of matching lines containing the string `FAIL`. Out of 500 total requests, exactly 100 resulted in failure.

Q2. How many different pages were requested, and which?

```
cut -d' ' -f5 access.log | sort -u | tee >(wc -l)
```

Output:

```
/page0
/page1
/page2
/page3
4
```

Explanation: The 5th whitespace-delimited field is extracted with `cut -d' ' -f5`, sorted and deduplicated using `sort -u`. The stream is split using `tee` and process substitution `>(wc -l)`, printing both the distinct page names and the cardinality (4).

Q3. How many requests did each page get?

```
cut -d' ' -f5 access.log | sort | uniq -c
```

Output:

```
    125 /page0
    125 /page1
    125 /page2
    125 /page3
```

Verification: The distribution is perfectly uniform ($125 \times 4 = 500$). The sum equals the total line count.

Q4. Which user or users have the most failed requests, and how many?

```
grep FAIL access.log | cut -d' ' -f3 | sort | uniq -c | sort -rn | awk 'NR==1{m=$1}
    $1==m'
```

Output:

```
     15 user5
     15 user3
```

Explanation: Failed lines are filtered with `grep FAIL`, the user column is extracted, sorted, counted with `uniq -c`, and sorted in descending order (`sort -rn`). The `awk` filter records the maximum count on the first row (`m=$1`) and prints all users matching this maximum value, correctly identifying that `user5` and `user3` tie for the most failures (15 each).

Q5. How many requests did user3 make, and how many of them failed?

```
grep ' user3 ' access.log | cut -d' ' -f4 | sort | uniq -c
```

Output:

```
     15 FAIL
     57 OK
```

Explanation: Exact field matching with spaces (`' user3 '`) prevents false substring matches. The 4th field (status) is extracted and counted, proving `user3` made $15+57 = 72$ total requests, with 15 failures.

Q6. Print the last 3 failed requests, showing only time and user.

```
grep FAIL access.log | tail -3 | cut -d' ' -f2,3
```

Output:

```
10:10 user0
10:15 user5
10:20 user3
```

Explanation: Filters for failure entries, extracts the final 3 occurrences using `tail -3`, and projects columns 2 (timestamp) and 3 (user identifier).

Q7. Which login shells appear in `/etc/passwd`, and how many accounts use each?

```
cut -d: -f7 /etc/passwd | sort | uniq -c | sort -rn
```

Output:

```
     26 /usr/bin/nologin
      3 /usr/bin/bash
      1 /usr/bin/git-shell
```

Explanation: Colon-separated field 7 in `/etc/passwd` defines the default user login shell. The pipeline tallies the accounts assigned to each shell in descending frequency.

Q8. Discrepancy between `ls /etc | wc -l` and `ls -l /etc | wc -l`

```
command ls /etc | wc -l ; command ls -l /etc | wc -l
```

Output:

```
192
193
```

Prediction and Explanation: The commands differ by exactly **1**. When `ls` detects that standard output is connected to a pipe, it suppresses columnar output and prints filenames one per line (192 entries). However, `ls -l` produces a long listing that begins with an introductory summary line indicating total disk block allocation (e.g., `total 1024`), followed by the 192 entry rows, yielding $192 + 1 = 193$ total lines.

3.3 Bonus Task (+1): Extended Regular Expressions

Filtering requests failing on ten-minute boundaries (10:00, 10:10, 10:20, 10:30, 10:40, 10:50):

```
grep -E ' 10:[0-5]0 .*FAIL' access.log
grep -E -c ' 10:[0-5]0 .*FAIL' access.log
```

Result: Exactly **50** requests matched the criteria.

3.4 Complete One-Liners Script (`oneliners.sh`)

The consolidated solution script `oneliners.sh` contains all pipeline solutions:

Listing 5: Contents of oneliners.sh

```
# Q1. How many requests failed?
grep -c FAIL access.log
# Q2. How many different pages were requested, and which?
cut -d' ' -f5 access.log | sort -u | tee >(wc -l)
# Q3. How many requests did each page get?
cut -d' ' -f5 access.log | sort | uniq -c
# Q4. Which user or users have the most failed requests, and how many?
grep FAIL access.log | cut -d' ' -f3 | sort | uniq -c | sort -rn | awk 'NR==1{m=$1}
    $1==m'
# Q5. How many requests did user3 make, and how many of them failed?
grep ' user3 ' access.log | cut -d' ' -f4 | sort | uniq -c
# Q6. Print the last 3 failed requests, showing only time and user.
grep FAIL access.log | tail -3 | cut -d' ' -f2,3
# Q7. Which login shells appear in /etc/passwd, and how many accounts use each?
cut -d: -f7 /etc/passwd | sort | uniq -c | sort -rn
# Q8. Why do ls /etc | wc -l and ls -l /etc | wc -l differ by exactly one?
command ls /etc | wc -l ; command ls -l /etc | wc -l
```

4 Script Engineering and Task Automation

4.1 Log Analysis Script Implementation (`report.sh`)

A shell script `report.sh` was implemented to automate summary generation from any access log:

Listing 6: The report.sh script implementation

```
#!/bin/bash
# usage: ./report.sh access.log
log=$1
if [ ! -f "$log" ]; then
    echo "no such file: $log" >&2
    exit 1
fi
total=$(wc -l < "$log")
fails=$(grep -c FAIL "$log")
echo "$total requests, $fails failed"
for n in 0 1 2 3 4 5 6; do
    c=$(grep " user$n " "$log" | grep -c FAIL)
    echo "user$n: $c failures"
done
```

4.2 Execution and Error Handling Verification

After granting execute permissions via `chmod +x report.sh`, the script was evaluated under normal and error conditions:

Listing 7: Normal execution on access.log

```
~ > ./report.sh access.log
```

```
500 requests, 100 failed
user0: 14 failures
user1: 14 failures
user2: 14 failures
user3: 15 failures
user4: 14 failures
user5: 15 failures
user6: 14 failures
```

Consistency Check: The individual user failures sum to $14 \times 5 + 15 \times 2 = 70 + 30 = 100$ total failures, confirming pipeline arithmetic integrity.

Testing robust error handling on an invalid file path:

Listing 8: Error handling on missing file

```
~ > ./report.sh nope.log ; echo "exit code: $?"
no such file: nope.log
exit code: 1
```

The script routed diagnostic text to `stderr` and terminated with exit code 1.

4.3 Bonus Task (+1): Automated Scheduling via Cron Daemon

To establish automated, recurring execution, a cron job was registered using `crontab`:

Listing 9: Crontab schedule entry

```
* * * * * /home/sunflower/os-lab3/report.sh /home/sunflower/os-lab3/access.log >>
    /home/sunflower/os-lab3/report.log 2>&1
```

The scheduling parameters `* * * * *` specify execution every minute. Both `stdout` and `stderr` were appended to `report.log`. After 130 seconds of execution, `report.log` was inspected:

Listing 10: Periodic output captured in report.log

```
500 requests, 100 failed
user0: 14 failures
user1: 14 failures
user2: 14 failures
user3: 15 failures
user4: 14 failures
user5: 15 failures
user6: 14 failures
500 requests, 100 failed
...
```

The log confirmed multiple automated executions. The crontab entry was subsequently purged using `crontab -r`, verified via `crontab -l`.

5 Answers to Laboratory Questions

5.1 Part 1 Questions

1. **Which of `cd`, `ls`, `echo` are built-ins, and why MUST `cd` be one?**
   Answer: `cd` and `echo` are built-ins. `ls` is an external binary located at `/usr/bin/ls`. `cd` **must** be a built-in because in the UNIX process model, child processes inherit

a copy of their parent's environment and working directory, but cannot alter the parent's environment. If `cd` were an external binary, the shell would fork a child process, the child would change its own working directory, and then exit. The parent shell's working directory would remain entirely unchanged.

2. **How many directories are in your PATH, and in which one is `ls`?**
   Answer: There are **15** directories in the system's `$PATH`. The `ls` executable is located in `/usr/bin`.

3. **What exit code does a command that is not found return?**
   Answer: It returns exit status **127**.

4. **Which option of `ls` sorts by modification time?**
   Answer: The `-t` option (or `-sort=time` in GNU coreutils, `-s date` in `eza`).

5.2 Part 2 Questions

1. **Differences in predictions vs. real output**:
   Documented in full in Table 2.

2. **`echo` knows nothing about files, yet `echo *.c` printed file names. Who produced them?**
   Answer: The shell (Bash) produced them. Before `echo` is executed, the shell performs pathname expansion (globbing). It scans the directory, matches the pattern `*.c`, replaces the token with the matching filenames, and passes them as discrete arguments in `argv` to `echo`.

3. **Why did `ls alfa beta.txt` fail? Give two different ways to fix it.**
   Answer: It failed because unquoted spaces are treated as argument delimiters (word splitting). The shell interpreted `alfa` and `beta.txt` as two separate non-existent files. Two fixes:

   - Quoting: `ls "alfa beta.txt"` or `ls 'alfa beta.txt'`
   - Escaping: `ls alfa\ beta.txt`

4. **What is the difference between single and double quotes? Why does `ls` display names with quotes?**
   Answer: Single quotes preserve literal character values and suppress all shell expansions. Double quotes allow variable expansion, command substitution, and arithmetic evaluation while preventing word splitting and globbing. `ls` displays files with spaces in quotes to alert the user that whitespace is part of the filename and prevent ambiguity.

5.3 Part 3 Questions

1. **Which file got the error message, and which got the listing?**
   Answer: `err.txt` received the error diagnostic (FD 2). `out.txt` received the standard directory listing (FD 1).

2. **Difference between `> all.txt 2>&1` and `2>&1 > all.txt`**
   Answer: Redirections are evaluated left to right. In `> all.txt 2>&1`, FD 1 is redirected to the file first, and then FD 2 is duplicated to FD 1 (writing both to the file). In `2>&1 > all.txt`, FD 2 is first duplicated to FD 1 (which still points to the screen), and only then is FD 1 redirected to the file. As a result, standard output goes to the file, while errors remain printed on the terminal screen.

3. **Why does `wc -l < access.log` print only a number, while `wc -l access.log` also prints the filename?**
   Answer: With a filename argument, `wc` reads the name from `argv` and outputs it for clarity. With stdin redirection (`<`), the shell connects the file directly to FD 0; `wc` reads from an anonymous stream and has no knowledge of any filename.

4. **Explain each operator in one sentence: `;`, `&&`, `||`**
   Answer:

   - `;` executes the succeeding command sequentially regardless of the preceding command's exit status.
   - `&&` executes the succeeding command only if the preceding command exited with status 0 (success).
   - `||` executes the succeeding command only if the preceding command exited with a non-zero status (failure).

5.4 Part 4 Question (Q8)

Why do `ls /etc | wc -l` and `ls -l /etc | wc -l` differ by exactly one?
Answer: When standard output is redirected into a pipe, `ls` emits exactly one filename per line without headers. In contrast, `ls -l` generates an introductory header line specifying total block allocation (e.g., `total 1024`), increasing the total line count by exactly 1.

5.5 Part 5 Questions

1. **What does the first line `#!/bin/bash` do, and what happens if you run `./report.sh` before `chmod +x`?**
   Answer: The shebang (`#!`) specifies the absolute path of the interpreter binary that the kernel must execute. Running the script before `chmod +x` fails with `Permission denied` (exit status 126) because the kernel enforces execute permission bit checks.

2. **Why must you type `./report.sh` and not just `report.sh`?**
   Answer: For security reasons, the current working directory (`.`) is not included in `$PATH` to prevent accidental or malicious execution of untrusted scripts. The `./` prefix provides an explicit relative path.

3. **Why does the script write the error with `>&2` and end with `exit 1`?**
   Answer: `>&2` routes error messages to standard error so they do not pollute standard output pipelines. `exit 1` returns a non-zero exit status signaling failure to calling processes and automation scripts.

6 Conclusions

In this laboratory work, the command-line interface, Bash shell mechanics, I/O stream redirection, composable text filtering pipelines, and shell scripting were comprehensively investigated and empirically verified.

Key takeaways and engineering insights include:

1. **Process Model and Shell Built-ins**: The execution model of UNIX requires commands affecting internal shell state (`cd`, `exit`) to run directly within the parent process as built-ins, whereas general system tools operate in isolated child processes created via `fork()` and `execve()`.

2. **Deterministic Expansion Ordering**: Understanding the sequence in which the shell interprets command lines (brace expansion, parameter expansion, command substitution, word splitting, and globbing) is critical to avoiding word splitting bugs and unintended wildcard matches.

3. **Stream Composability and Redirection**: Linux treats I/O streams as clean byte conduits. Masterful control over file descriptors 0, 1, and 2 enables seamless diagnostic isolation and data preservation. The evaluation order of redirection operators (e.g., `> all.txt 2>&1`) is strictly sequential from left to right.

4. **Power of the UNIX Pipeline**: Complex data analysis questions across hundreds of log records can be solved with elegant one-liners combining basic tools (`grep`, `cut`, `sort`, `uniq`, `awk`).

5. **Production Automation**: Encapsulating workflows into shell scripts with parameter verification, standard error handling, appropriate exit codes, and cron automation provides a foundation for modern systems engineering and DevOps automation.